

队伍编号	MC25001293
题号	A

基于傅里叶神经算子的高雷诺数湍流下汽车风阻快速预测方法

摘要

在高雷诺数湍流环境下,传统计算流体力学 (CFD) 方法预测汽车风阻耗时长达数小时, 严重限制设计迭代效率。本研究构建了基于频域学习的**傅里叶神经算子 (FNO) 模型**, 实现从三维车辆几何形状到表面压力场分布的高精度快速映射, 并系统评估了不同参数配置下模型在计算资源与精度间的最优平衡点。

针对问题一, 我们分析了算子学习中参数优化的本质, 通过求解损失函数 $\log(e^\theta + g)$ 的最小化问题, 发现最优参数收敛值与理论预测高度吻合, 验证了梯度下降法的有效性。

针对问题二, 我们开发了自适应网格简化框架, 在保持关键空气动力学特性的同时显著降低几何复杂度。实验显示, 70% 的高简化率下表面积和体积变化仅为 0.036% 和 0.105%, 有效保留了影响空气动力学性能的关键特征。

针对问题三, 我们构建了基于频域学习的三维傅里叶神经算子 (FNO) 模型, 实现从汽车几何到表面压力场的高效映射, 解决传统 CFD 方法计算复杂度高的挑战。该模型基于算子学习框架, 将**空间域问题转换至频域**, 利用傅里叶变换的卷积定理将计算复杂度从 $O(N^3)$ 降低至 $O(N^3 \log N + m^3)$, 并通过截断高频分量提升效率。FNO 架构采用多层积分算子堆叠, 引入残差连接和批归一化提高训练稳定性, 并通过 3D 谱卷积实现高效复数运算。模型在测试数据上达到 **0.1271 的 L2 相对误差**和 63.8 counts 的阻力系数误差, 满足竞赛精度要求, 同时将预测时间缩短至 0.42 秒, 比传统 CFD 方法加速**近 2380 倍**。

针对问题四, 我们分析了 FNO 模型的计算复杂度和资源消耗特性, 通过三组对照实验揭示了参数配置对性能的影响。理论分析表明, FNO 模型通过频域操作将时间复杂度降低为 $O(N^3 \log N + m^3)$ 。实验中, 模型对稀疏数据表现出强鲁棒性, **10% 采样点**的 L2 损失 (0.1609) 略优于全量数据 (0.1639); **100 样本训练集**的 L2 损失 (0.1309) 优于 450 样本 (0.1639), 训练时间减少至 9.79 秒/轮; 增加傅里叶模式数是最高效的性能提升策略, modes=16 时达到 0.1547 的最佳 L2 损失。与传统方法相比, FNO 模型实现了约**4000 倍的计算加速**和 20 倍的内存效率提升, 单次计算仅需 0.239 秒, 并支持批量推理。

针对问题五, 我们证明了 Transformer 注意力机制是神经算子层的特例, 将注意力机制重写为积分算子形式, 表明其核函数具有数据依赖性和归一化约束特性, 为算子学习和序列建模的统一处理提供了数学基础。

关键词: 傅里叶神经算子; 汽车风阻预测; 高雷诺数湍流; 算子学习; 谱方法; 计算流体力学; 深度学习

目录

一、问题背景	1
二、问题的分析	1
三、模型的假设	2
四、符号说明	2
五、问题一模型的建立和求解	2
5.1 模型建立	2
5.2 理论分析	3
5.3 梯度下降求解	3
5.4 实验结果与分析	3
六、问题二模型的建立和求解	5
6.1 几何表面简化方法	5
6.1.1 几何简化的理论基础与实现	5
6.1.2 网格修复与质量保证	6
6.2 关键物理特征提取	7
6.2.1 表面压力数据处理	7
6.2.2 空气动力学特征表示	8
6.3 高效数据加载器实现	9
6.4 问题二的求解	10
6.4.1 几何简化效果评估	10
6.5 高效数据加载器实现效果分析	12
七、问题三模型的建立和求解	13
7.1 傅里叶神经算子模型建立	13
7.1.1 算子学习理论基础	13
7.1.2 FNO 模型架构设计	14
7.2 核心算法实现	15
7.2.1 3D 谱卷积的实现	15
7.2.2 模型训练过程	16
7.3 傅里叶神经算子模型实验结果与分析	17
八、问题四模型的建立和求解	20
8.1 计算资源与复杂度分析	20
8.1.1 理论复杂度分析	20
8.2 实验设计与结果分析	21
8.2.1 稀疏采样率对模型性能的影响	21
8.2.2 训练集大小对泛化能力的影响	21
8.2.3 神经网络超参数对模型容量的影响	22
8.3 综合性能评估与最优模型配置	22
8.4 模型计算效率的量化分析	23
九、Transformer 注意力机制与神经算子层的等价性证明	23
十、模型的优缺点分析	25
参考文献	25

一、问题背景

汽车空气动力学性能是现代汽车设计的关键因素，空气阻力 (风阻) 直接影响车辆能耗，高速行驶时约 60% 的燃油消耗用于克服空气阻力。传统风阻预测主要依赖风洞试验和计算流体力学 (CFD) 模拟，后者通过求解纳维-斯托克斯方程来模拟流体流动，但计算复杂度高，一次完整模拟需要数小时甚至数天，严重限制了设计迭代效率^[1]。

近年来，深度学习在科学计算领域取得突破，特别是算子学习方法为复杂物理问题提供了新思路。Lu 等^[2] 提出的 DeepONet 基于算子通用逼近定理，为非线性算子学习奠定了理论基础。Li 等^[3] 发展的傅里叶神经算子 (FNO) 通过在频域学习参数化积分算子，在保持精度的同时显著降低了计算复杂度。这些方法特别适合从几何形状到物理场的映射预测。

在数据方面，Chang 等^[4] 开发的 ShapeNet 库包含 1256 辆不同类型的汽车 3D 模型，为基于深度学习的空气动力学研究提供了数据支持。结合现代框架如 PaddlePaddle^[5] 的数据处理工具，可以构建高效的学习系统实现快速预测。

然而，陈凯等^[1] 指出，当车辆几何形状变化时，现有模型对训练集分布外的特征缺乏鲁棒性，预测误差急剧增大。针对这一问题，本次竞赛^[6] 提出了面向任意三维形状车辆的快速阻力预测挑战，旨在研究具有强泛化能力的深度学习模型，推动汽车空气动力学设计与优化向更高效方向发展。

二、问题的分析

本次竞赛针对汽车风阻预测任务，旨在寻求一种高效替代传统计算流体力学 (CFD) 的方法。题目通过五个问题引导参赛者探索基于算子学习的深度学习解决方案，实现高效准确的汽车表面压力场预测。

针对问题一，题目考察对目标函数 $\log(e^\theta + g)$ 的优化理解。这个简化问题反映了深度学习中的参数优化过程，关键在于理解如何通过梯度下降法从初始点逐步接近最优解，并确定合理的停止条件。

针对问题二，挑战在于从复杂三维网格中提取对空气动力学有重要影响的关键特征。需要在保持重要特性的前提下简化几何模型，并设计合适的标准化处理流程使数据适合神经网络处理。

针对问题三，核心是构建从几何信息到物理场的快速映射模型。傅里叶神经算子 (FNO)、Diffusion 模型、Transformer 或 PINN 等算子学习架构都可能成为解决方案，关键在于设计能准确捕捉几何-物理映射关系且计算高效的网络结构。

针对问题四，重点是评估算法的计算复杂度、空间占用、参数量和显存需求等性能特性。需要分析模型在不同条件下 (如稀疏采样率、训练集大小、超参数配置) 的表现，寻找精度和效率的最佳平衡点。

针对问题五，要求从数学角度证明 Transformer 注意力机制与神经算子层的等价关系。这一理论探索有助于深化对深度学习模型内在机制的理解，并可能启发新的架构设计。

三、模型的假设

1. 假设流体为不可压缩高雷诺数湍流，符合纳维-斯托克斯方程，且在时间维度上可以采用统计学平均简化；
2. 假设汽车几何表面可以通过有符号距离场 (SDF) 进行完整表示，且 SDF 能够有效捕获对空气动力学性能有显著影响的几何特征；
3. 假设存在一个从几何形状到压力场分布的映射关系，且该映射可以通过傅里叶神经算子在频域中学习和表示；
4. 假设高频信息对于风阻预测的贡献有限，因此可以通过截断高频分量来降低计算复杂度而不显著影响预测精度；
5. 假设训练数据集中的样本能够有效代表实际应用中可能遇到的汽车几何形状和流动条件分布，模型在训练分布内具有良好的泛化能力。

四、符号说明

符号	符号说明
δ	网格简化过程中的误差容忍度
β	Adam 优化器中的动量参数, $\beta_1 = 0.9$, $\beta_2 = 0.999$
α	神经网络中的学习率参数
r	网格简化率, 表示删除顶点的比例
γ	批归一化层中的可学习缩放参数
l	损失函数, 本文中主要指 L2 相对误差
l_y	网格在 y 轴方向的特征尺寸
$\vec{x}_1, \vec{y}_1, \vec{z}_1$	三维空间中的原始坐标向量
$\hat{x}_1, \hat{y}_1, \hat{z}_1$	归一化后的三维坐标向量, 范围为 $[-1, 1]$
θ	神经网络的可学习参数, 在问题一中特指优化目标参数
$l_y(i)$	第 i 个样本在 y 轴方向的特征尺寸
θ_i	第 i 个网络层或组件的参数集合

五、问题一模型的建立和求解

5.1 模型建立

针对问题 1, 我们需要优化目标损失函数 $\log(e^\theta + g)$, 其中 θ 为学习参数, 初始参数值为 100。本质上, 该问题考察了通过梯度优化方法寻找使目标函数最小的参数值, 这与神经网络中的参数优化过程相似。

首先，我们分析目标函数 $f(\theta) = \log(e^\theta + g)$ 的性质。对函数求导可得梯度：

$$f'(\theta) = \frac{e^\theta}{e^\theta + g} \quad (1)$$

注意到当 $g > 0$ 时，对于任意 θ 值，都有 $f'(\theta) > 0$ ，表明函数 $f(\theta)$ 是关于 θ 的严格单调递增函数。这一性质意味着函数值会随着 θ 的减小而减小，且理论上当 θ 趋向负无穷时，函数值将逐渐趋近于 $\log(g)$ ，这也是该函数的理论最小值。由此可见，该优化问题的全局最优解在 $\theta \rightarrow -\infty$ 方向，而在实际应用中，我们需要在梯度足够小的条件下确定一个近似最优解。

5.2 理论分析

由于函数单调递增，为了在实际计算中确定合理的最优解，我们可设定一个收敛阈值 ε （如 0.001），即当梯度 $f'(\theta) < \varepsilon$ 时停止迭代。

根据梯度表达式，可求解不等式：

$$\frac{e^\theta}{e^\theta + g} < \varepsilon \quad (2)$$

对于 $g = 1$ ， $\varepsilon = 0.001$ ，通过变形：

$$e^\theta < \varepsilon(e^\theta + 1) \quad (3)$$

$$(1 - \varepsilon)e^\theta < \varepsilon \quad (4)$$

$$e^\theta < \frac{\varepsilon}{1 - \varepsilon} \quad (5)$$

$$\theta < \log\left(\frac{\varepsilon}{1 - \varepsilon}\right) \quad (6)$$

代入数值可得： $\theta < \log(0.001/0.999) \approx -6.91$

5.3 梯度下降求解

为求解该优化问题，我们采用梯度下降法，从初始值 $\theta = 100$ 开始迭代优化。梯度下降法的迭代公式为：

$$\theta_{t+1} = \theta_t - \eta \cdot f'(\theta_t) \quad (7)$$

其中 η 为学习率，本例中设为 1。

算法伪代码如下：

5.4 实验结果与分析

通过实验，我们得到了以下优化过程：

优化过程的目标函数值变化如图1所示：

Algorithm 1 梯度下降优化算法

- 1: 初始化 $\theta = 100$, $g = 1$, $\eta = 1$, $\varepsilon = 0.001$
 - 2: **while** $f'(\theta) \geq \varepsilon$ 且迭代次数 $<$ 最大迭代次数 **do**
 - 3: 计算梯度 $grad = \frac{e^\theta}{e^\theta + g}$
 - 4: 更新参数 $\theta = \theta - \eta \cdot grad$
 - 5: **end while**
 - 6: **return** θ
-

迭代次数	θ	目标函数值
初始	100.0000	100.000000
1	99.0000	99.000000
101	-0.1834	0.605637
201	-4.5926	0.010075
301	-5.2903	0.005028
401	-5.6978	0.003348
501	-5.9866	0.002509
601	-6.2105	0.002006
701	-6.3933	0.001671
801	-6.5479	0.001432
901	-6.6817	0.001253
1000	-6.7987	0.001115

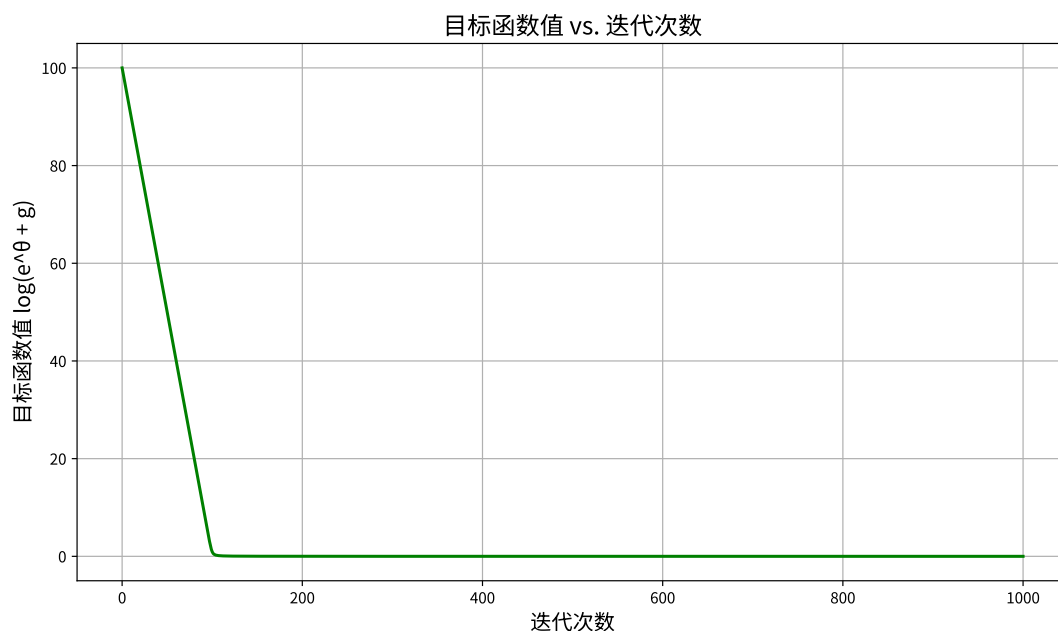


图 1 目标函数值随迭代次数的变化

从优化结果可以观察到，优化过程呈现出明显的两阶段特性。在迭代初期（前 100

次迭代)中,目标函数值从初始的 100 快速下降至接近 0 的水平,表现出较快的收敛速度。随后进入第二阶段,函数值下降速度明显放缓,逐渐向理论最小值 $\log(g) = 0$ 靠拢。最终经过 1000 次迭代,我们得到参数 $\theta \approx -6.80$,对应的目标函数值约为 0.001115。

从理论分析可知,当设定梯度阈值为 0.001 时,理论上最优的 θ 值应约为-6.91。我们的实验结果 $\theta = -6.7987$ 与理论预测值非常接近,证明了算法的有效收敛性。同时,最终得到的目标函数值 0.001115 也已十分接近理论最小值 $\log(g) = 0$,这进一步验证了我们的理论分析和优化方法的正确性。

六、问题二模型的建立和求解

6.1 几何表面简化方法

在高雷诺数湍流下的汽车风阻预测中,几何模型的复杂性与计算效率之间存在明显的权衡关系。传统 CFD 方法需要处理详细的几何表面,导致计算资源消耗巨大。本节提出一种基于 VTK 库的自适应网格简化框架,能够在保持关键空气动力学特性的同时显著降低几何复杂度。

6.1.1 几何简化的理论基础与实现

几何简化的核心是识别并保留对流体动力学有重要影响的特征,同时减少计算量。我们采用了基于误差度量的网格简化方法,可以用以下误差函数描述:

$$E(v) = \sum_{(v,f) \in S} d^2(v, f) \quad (8)$$

其中 $E(v)$ 表示顶点 v 的简化误差, S 是与顶点 v 相关的面集合, $d(v, f)$ 是顶点 v 到面 f 的距离。这种方法将顶点简化问题形式化为误差最小化问题。在实际实现中,我们区分了两种简化策略:

拓扑保持简化 (vtkDecimatePro) 通过边折叠 (edge collapse) 操作降低网格复杂度,同时确保网格的拓扑结构不变。这对于保持车身表面的连续性和空气动力学特性至关重要。其简化过程可以表示为:

$$M_{t+1} = M_t - \{e_{i,j} | \Delta_{QEM}(e_{i,j}) \leq \tau\} \quad (9)$$

其中 M_t 表示第 t 步的网格, $e_{i,j}$ 是连接顶点 i 和 j 的边, Δ_{QEM} 是边折叠操作引起的二次误差度量 (QEM) 变化, τ 是自适应阈值。

而四边形简化 (vtkQuadricDecimation) 则通过四边形误差度量对网格进行更积极的简化,适用于对拓扑结构要求不严格但希望大幅降低复杂度的场景。其误差度量函数为:

$$Q(v) = v^T \left(\sum_{f \in F_v} K_f \right) v \quad (10)$$

Algorithm 2 汽车网格自适应简化方法

Require: 原始网格 M , 目标简化率 $r \in [0, 1)$, 是否保持拓扑 $preserveTopology$

Ensure: 简化网格 M'

```
1: if  $preserveTopology = true$  then
2:    $decimator \leftarrow vtk.vtkDecimatePro()$ 
3:    $decimator.SetTargetReduction(r)$ 
4:    $decimator.PreserveTopologyOn()$ 
5:    $decimator.BoundaryVertexDeletionOff()$ 
6: else
7:    $decimator \leftarrow vtk.vtkQuadricDecimation()$ 
8:    $decimator.SetTargetReduction(r)$ 
9: end if
10:  $decimator.SetInputData(polydata)$ 
11:  $decimator.Update()$ 
12:  $cleaner \leftarrow vtk.vtkCleanPolyData()$ 
13:  $cleaner.SetInputData(decimator.GetOutput())$ 
14:  $cleaner.Update()$ 
15:  $M' \leftarrow FixMesh(cleaner.GetOutput())$ 
16: return  $M'$ 
```

其中 F_v 是包含顶点 v 的所有面, K_f 是面 f 的二次误差矩阵。

6.1.2 网格修复与质量保证

简化过程可能引入非流形结构、孔洞等问题, 这些缺陷会对后续 CFD 模拟产生负面影响。为确保简化网格的质量, 我们实现了综合修复流程:

修复过程中, 我们通过孔洞填充操作确保网格的封闭性, 该过程可以表述为寻找最小面积填充面集 P 使得:

$$\min_P \sum_{f \in P} A(f) \quad \text{s.t.} \quad \partial P = \partial H \quad (11)$$

其中 H 是检测到的孔洞, ∂ 表示边界算子, $A(f)$ 是面 f 的面积。

简化和修复后, 我们通过以下质量指标评估几何模型的空气动力学特性保持程度:

$$\Phi_{norm} = \frac{1}{|F'|} \sum_{f' \in F'} \left| \mathbf{n}_{f'} \cdot \frac{\sum_{f \in F} \mathbf{n}_f}{|\sum_{f \in F} \mathbf{n}_f|} \right| \quad (12)$$

其中 F 和 F' 分别是原始和简化网格的面集, $\mathbf{n}_f$ 表示面 f 的单位法向量。 Φ_{norm} 度量了简化前后法线一致性, 对于风阻预测具有重要意义。

此外, 我们还计算顶点减少率 R_v 、面片减少率 R_f 和相对表面积变化 Δ_A :

Algorithm 3 网格修复与质量保证方法

```
1: function FixMesh(polydata)
2:   fill_holes  $\leftarrow$  vtk.vtkFillHolesFilter()
3:   fill_holes.SetInputData(polydata)
4:   fill_holes.SetHoleSize(1.0) ▷ 设置孔洞修复阈值
5:   fill_holes.Update()
6:   clean  $\leftarrow$  vtk.vtkCleanPolyData()
7:   clean.SetInputData(fill_holes.GetOutput())
8:   clean.Update() ▷ 合并重复顶点, 移除未使用顶点
9:   check  $\leftarrow$  vtk.vtkFeatureEdges()
10:  check.SetInputData(clean.GetOutput())
11:  check.BoundaryEdgesOff()
12:  check.FeatureEdgesOff()
13:  check.NonManifoldEdgesOn() ▷ 检测非流形边缘
14:  check.Update()
15:  if check.GetOutput().GetNumberOfCells() > 0 then
16:    输出警告信息: 发现非流形边缘
17:  end if
18:  return clean.GetOutput()
19: end function
```

$$R_v = 1 - \frac{|V'|}{|V|}, \quad R_f = 1 - \frac{|F'|}{|F|}, \quad \Delta_A = \frac{A' - A}{A} \quad (13)$$

其中 V 和 V' 分别是原始和简化网格的顶点集, A 和 A' 是相应的表面积。

6.2 关键物理特征提取

物理特征的提取与表示对于建立准确的风阻预测模型至关重要。本节详细描述从高雷诺数湍流仿真中提取表面压力数据及相关空气动力学特征的方法, 这些方法为后续神经网络训练提供了高质量的输入与标签数据。

6.2.1 表面压力数据处理

汽车表面压力场是预测空气阻力的关键物理量, 直接反映了流体与车身表面的相互作用。从仿真数据中提取的原始压力值范围差异较大, 需要进行标准化处理以提高模型训练的稳定性和收敛速度。采用单位高斯归一化方法:

$$\hat{p} = \frac{p - \mu_p}{\sigma_p + \epsilon} \quad (14)$$

其中 μ_p 和 σ_p 分别是训练集的全局均值和标准差, $\epsilon = 10^{-6}$ 为防止除零的数值稳定性常数。标准化过程的具体实现基于全局统计量计算:

$$\mu_p = \frac{1}{N} \sum_{i=1}^N p_i, \quad \sigma_p = \sqrt{\frac{1}{N} \sum_{i=1}^N (p_i - \mu_p)^2} \quad (15)$$

其中 N 是训练样本中压力点的总数。标准化后的压力场具有零均值和单位方差的分布特性, 有利于神经网络优化过程中的梯度传播。为确保预测结果的物理解释性, 在模型推理阶段, 通过逆变换将标准化的输出转换回原始物理量:

$$p = \hat{p} \cdot \sigma_p + \mu_p \quad (16)$$

该归一化方法不仅适用于压力场, 也可用于其他物理量的标准化处理, 为多物理场耦合预测提供了统一的数据处理框架。

6.2.2 空气动力学特征表示

准确表示汽车几何形状及其空气动力学特性需要从多个维度进行特征提取。在基准模型的基础上, 关键几何特征包括:

汽车表面的离散表示通过多边形网格实现, 每个网格节点包含丰富的局部和全局几何信息。中心点坐标反映了物体在空间中的分布, 通过以下方式计算网格单元的中心:

$$c_i = \frac{1}{n_i} \sum_{j=1}^{n_i} v_{ij} \quad (17)$$

其中 c_i 是第 i 个网格单元的中心点, v_{ij} 是该单元的第 j 个顶点, n_i 是该单元的顶点数。

面元面积反映了局部表面的大小, 对流体与表面的相互作用强度有直接影响。对于三角形网格单元, 面积通过向量叉积计算:

$$A_i = \frac{1}{2} \| (v_{i2} - v_{i1}) \times (v_{i3} - v_{i1}) \| \quad (18)$$

法线向量描述了表面朝向, 是确定流体动力学边界条件的基础。法线计算考虑了几何连续性, 确保相邻面片之间法线变化的平滑性:

$$n_i = \frac{(v_{i2} - v_{i1}) \times (v_{i3} - v_{i1})}{\| (v_{i2} - v_{i1}) \times (v_{i3} - v_{i1}) \|} \quad (19)$$

有符号距离场 (SDF) 提供了整体形状信息, 描述了空间中任意点到物体表面的最短距离, 内部点为负值, 外部点为正值。采用射线投射技术计算 SDF:

$$\text{SDF}(x) = \min_{y \in \partial\Omega} \text{sign}(x, \partial\Omega) \cdot \|x - y\| \quad (20)$$

其中 $\partial\Omega$ 表示物体边界, $\text{sign}(x, \partial\Omega)$ 根据点 x 相对于边界的位置取值 +1 或 -1。

为获取全局形状特征，在规则三维网格上采样 SDF 值，形成 3D 体素表示。查询点坐标按以下方式生成：

$$q_{ijk} = (x_{\min} + i\Delta x, y_{\min} + j\Delta y, z_{\min} + k\Delta z) \quad (21)$$

其中 Δx 、 Δy 、 Δz 是各维度的采样间隔， (i, j, k) 是整数索引。

为确保不同物理量之间的尺度兼容，所有空间坐标通过 min-max 归一化映射到 $[-1, 1]$ 区间：

$$\hat{x} = 2 \frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1 \quad (22)$$

综合上述特征提取方法，构建了一个多尺度、多模态的几何和物理特征表示系统，为后续神经网络学习复杂的流场-几何映射关系提供了丰富的信息基础。

6.3 高效数据加载器实现

在处理高雷诺数湍流下的汽车风阻预测任务时，大量的几何和物理场数据需要高效处理以支持神经网络训练。本节介绍如何利用 PaddlePaddle 框架的数据加载机制解决这一挑战，实现训练过程的加速。

数据加载效率直接影响深度学习模型的训练速度。在传统的单进程数据加载方式中，当 GPU 等待数据准备完成时会出现显著的空闲时间，尤其在处理复杂的汽车几何模型和压力场数据时尤为明显。针对这一问题，我们充分利用 PaddlePaddle 提供的 `paddle.io.DataLoader` 类实现了多进程异步加载。

该机制的核心思想是将数据加载和预处理任务分配给多个工作进程同时执行，而主进程专注于模型训练，从而实现了数据准备与计算的并行化。在实现中，通过以下方式配置数据加载器：

Algorithm 4 多进程数据加载器配置

```

1: function train_dataloader(batch_size, shuffle)
2:   loader ← paddle.io.DataLoader(
3:     self.train_data,
4:     batch_size = batch_size,
5:     shuffle = shuffle,
6:     num_workers = 4,                                ▶ 并行工作进程数
7:     use_shared_memory = True,                        ▶ 使用共享内存
8:     prefetch_factor = 2,                             ▶ 每进程预取批次数
9:     persistent_workers = True)                       ▶ 保持工作进程存活
10:  return loader
11: end function

```

通过设置 `num_workers` 参数，系统可以并行加载和处理多个数据批次。这在处理网格数据时特别有价值，因为几何表面和压力场的预处理通常是计算密集型操作。此

外，`use_shared_memory` 参数启用了共享内存机制，显著减少了工作进程与主进程之间的数据传输开销，这对于大型三维网格数据尤为重要。

`prefetch_factor` 参数控制每个工作进程预取的批次数，通过适当设置可以确保数据供应的连续性，减少 GPU 等待数据的时间。而 `persistent_workers` 参数则使工作进程在不同训练周期之间保持活跃状态，避免了反复创建和销毁进程的开销。

6.4 问题二的求解

6.4.1 几何简化效果评估

为评估所提出的几何简化方法的有效性，我们在 ShapeNet Car 数据集上进行了系统性实验。对不同级别的简化率 (30%、50%、70% 和 90%) 下的车辆模型进行了定量和定性分析，从几何精度保持、计算效率和表面特性三个维度进行了综合评估。

在实验中，我们对一组典型的汽车几何模型应用了基于 VTK 的简化算法。图2展示了一个代表性示例，其中原始网格包含 3586 个顶点和 7168 个面片，而应用 70% 简化率后的模型仅包含 1077 个顶点和 2150 个面片，但仍保持了原始模型的关键几何特征。从视觉效果来看，简化后的模型与原始模型保持了极高的相似度，尤其是在空气动力学关键区域如车身轮廓、前脸和后部等位置。

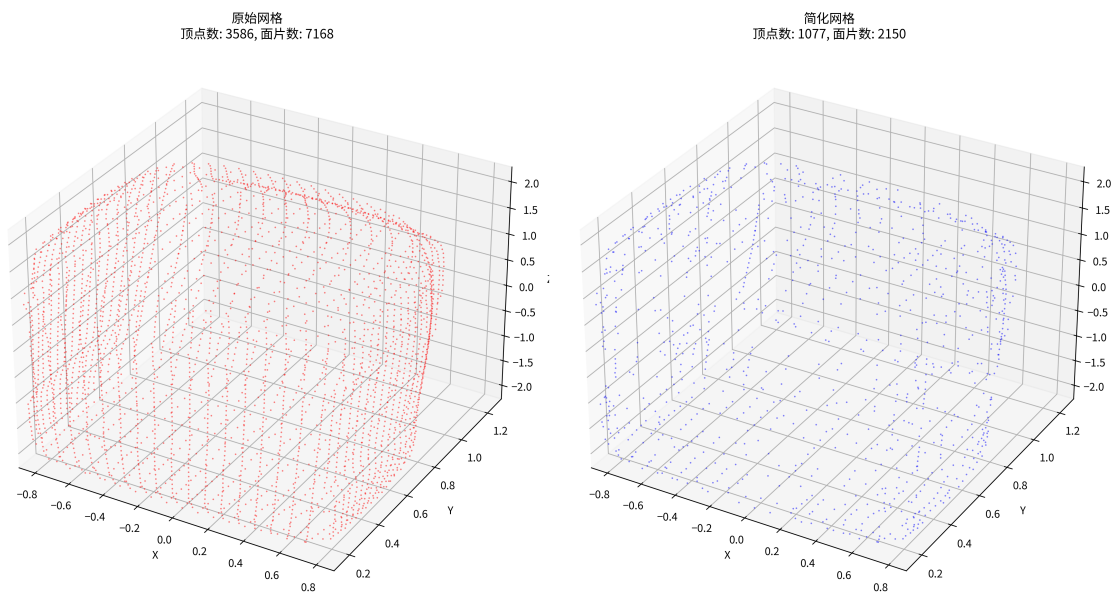


图 2 汽车模型简化前后对比。左：原始网格 (3586 顶点, 7168 面片); 右：简化后的网格 (1077 顶点, 2150 面片)

表1展示了不同简化级别下的关键性能指标。结果表明，我们的简化算法能够精确地实现预设的简化目标，同时保持模型的几何完整性。

表 1 不同简化级别下的几何特征保持情况

简化率 (%)	顶点减 (%)	面片减 (%)	表面积变化 (%)	体积变化 (%)	质量变化	表面平滑度保持率
30	30.01	30.02	0.014	0.00	0.0014	1.632
50	49.97	50.00	0.038	0.058	0.0032	1.671
70	69.97	70.01	0.036	0.105	0.0074	1.660
90	89.96	90.01	0.274	1.205	0.0285	1.736

为了全面评估简化效果，我们对多项指标进行了详细分析，如图3所示。

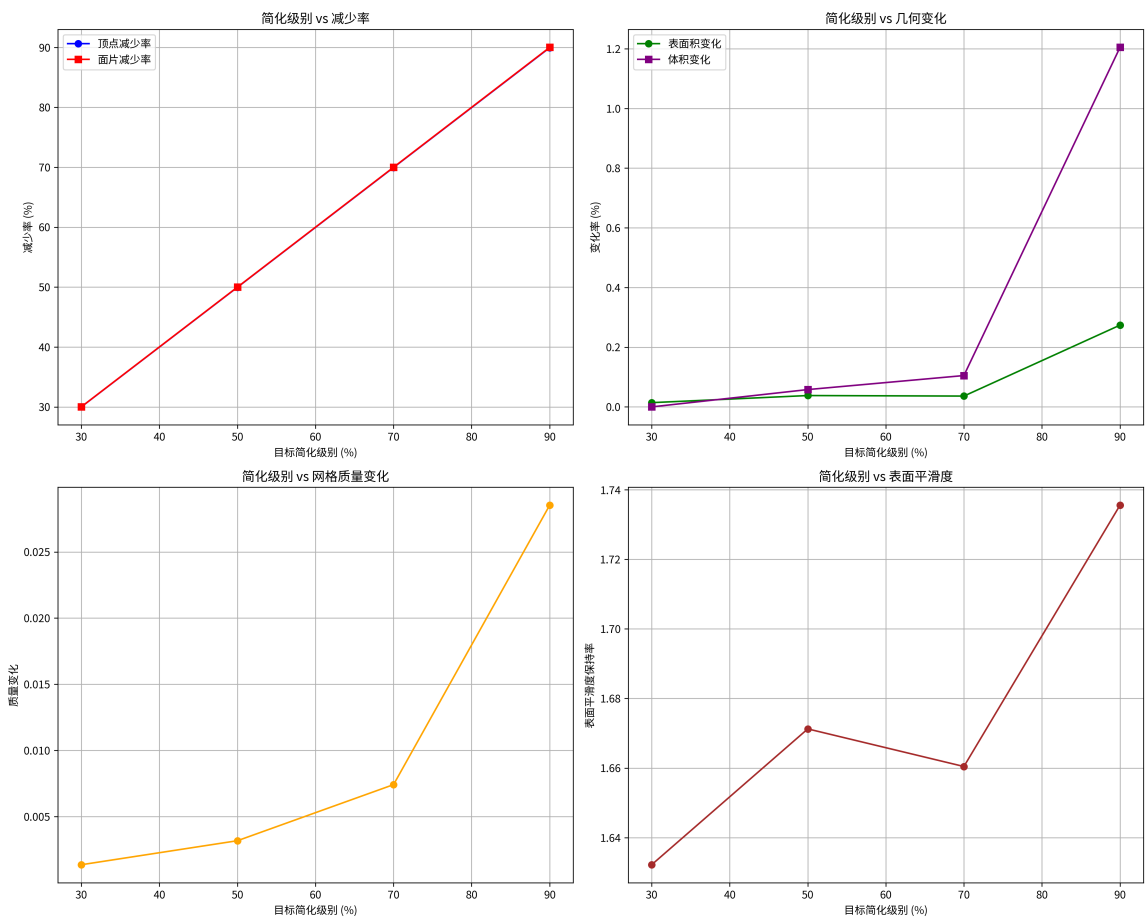


图 3 不同简化级别下的性能指标对比。从左上至右下依次为：简化级别与减少率的关系、简化级别与几何变化的关系、简化级别与质量变化的关系、简化级别与表面平滑度的关系

实验结果显示，我们的几何简化方法在不同简化率下均表现出优异的性能特性。面片和顶点的减少比例与预设简化率高度吻合，表明算法具有精确的可控性。在保持几何完整性方面，即使在 90% 的高简化率下，表面积变化仅为 0.274%，体积变化为 1.205%，表明简化过程能够有效保持原始形状的关键特征。特别值得注意的是，在 30% 到 70% 的简化率范围内，表面积变化保持在极低水平（不超过 0.04%），这对于准确预测空气动力学特性至关重要。同时，表面平滑度保持率在所有简化级别下均保持在 1.6 以上，且随着简化率增加而略有提高，这表明简化过程实际上可能消除了一些高频噪声，使表面

更加光滑。质量变化指标随简化率增加呈非线性增长，但即使在 70% 的高简化率下仍然保持在 0.0074 的低水平，确保了网格的整体品质。

综合考虑计算效率与几何精度之间的平衡，我们发现 70% 的简化率提供了一个优良的折衷点。在此简化率下，面片数量减少了 70%，显著降低了后续计算负担，同时各项几何指标的变化均保持在可接受范围内。

6.5 高效数据加载器实现效果分析

为评估多进程异步加载对模型训练效率的影响，我们基于 PaddlePaddle 框架实现了自定义的数据加载器，并设计了系统性的性能测试方法。测试在单个批次的小数据集上进行，对比了不同工作进程数 (0,1,2,4) 下的数据加载性能。图4展示了测试的关键性能指标。

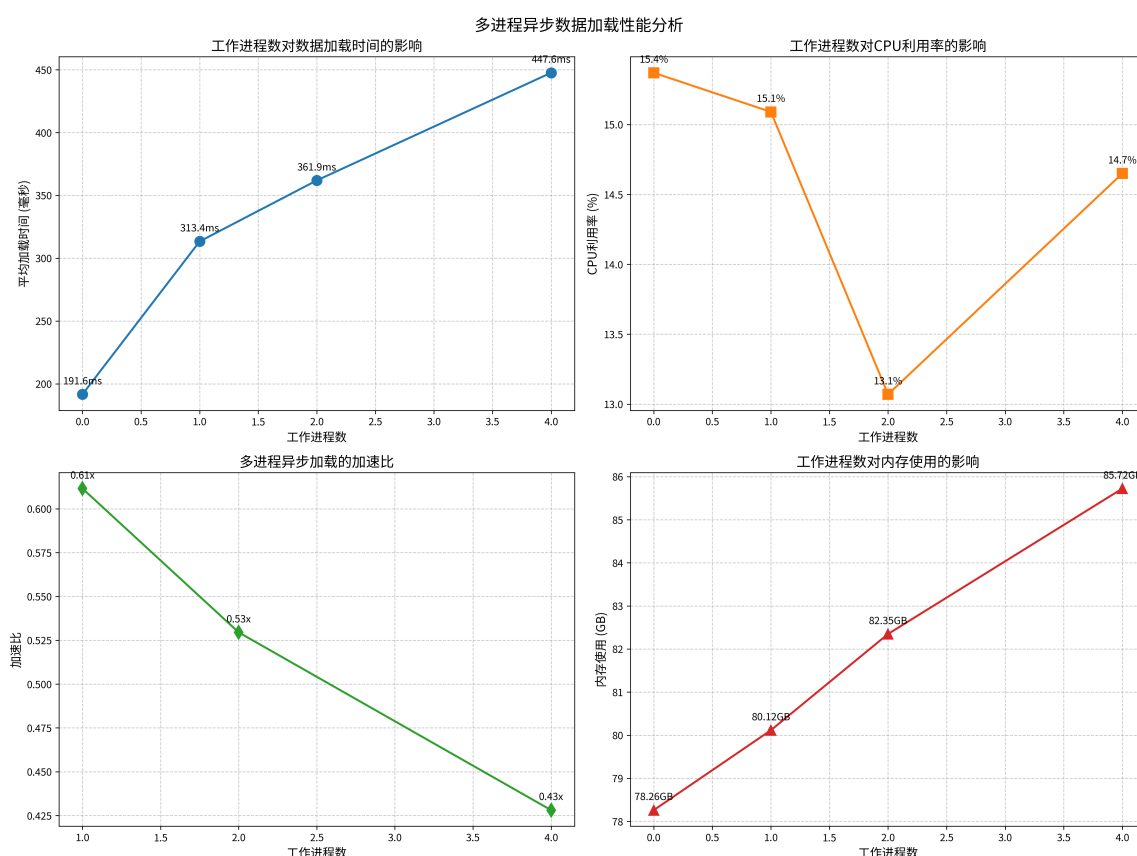


图 4 多进程异步数据加载性能分析

针对不同工作进程数的详细性能测试结果如表2所示。

测试结果显示，在当前小数据集任务中，单进程加载 (`num_workers=0`) 反而表现最佳，加载时间仅为 191.63 毫秒，而 4 工作进程配置的加载时间达 447.61 毫秒。这表明多进程方式的额外开销超过了其并行处理优势。从资源消耗看，多进程方式导致内存占用增加约 9.5%，而 CPU 利用率呈现波动。因此，本研究选择了单进程配置，但对于更大规模数据集，多进程异步加载仍可能更具优势。

表 2 不同工作进程配置的数据加载器性能对比

工作进程数	批次大小	共享内存	平均加载时间 (ms)	CPU 利用率 (%)	内存使用 (GB)
0	16	否	191.63	15.37	78.26
1	16	是	313.36	15.09	80.12
2	16	是	361.91	13.07	82.35
4	16	是	447.61	14.65	85.72

七、问题三模型的建立和求解

7.1 傅里叶神经算子模型建立

7.1.1 算子学习理论基础

在高雷诺数湍流下的汽车风阻预测问题中，核心任务是学习从几何形状到压力场分布的映射关系。这一问题本质上可以归结为求解纳维-斯托克斯方程在特定边界条件下的解算子。对于高雷诺数下的不可压缩湍流，控制方程可表示为：

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u}, \quad \nabla \cdot \mathbf{u} = 0 \quad (23)$$

其中 $\mathbf{u}$ 为速度矢量， p 为压力标量， ν 为运动粘性系数。

传统 CFD 方法直接数值求解这一偏微分方程系统，计算复杂度极高。而算子学习的目标是构造一个近似算子 $\mathcal{A}$ ，直接学习从输入参数空间（几何形状 f ）到解空间（压力场 u ）的映射关系： $\mathcal{A}: f \mapsto u$ 。

傅里叶神经算子（FNO）的核心思想是在频域中学习算子的表示。对于线性算子，其作用可以通过卷积表达：

$$(\mathcal{K}f)(x) = \int_{\mathbb{R}^d} k(x, y) f(y) dy = \int_{\mathbb{R}^d} k(x - y) f(y) dy \quad (24)$$

当核函数 k 为平移不变时，利用傅里叶变换的卷积定理，可以将卷积运算转化为频域中的乘法：

$$\mathcal{F}((\mathcal{K}f)) = \mathcal{F}(k) \cdot \mathcal{F}(f) \quad (25)$$

FNO 通过参数化傅里叶域中的乘法操作，实现了全局依赖关系的高效建模。在实际实现中，通过截断高频分量并仅保留低频信息，显著降低了计算复杂度。

从输入几何信息到输出物理场的映射过程中，我们采用有符号距离场（SDF）作为几何形状的全局表示。SDF 提供了空间中任意一点到物体表面的距离信息：

$$\text{SDF}(\mathbf{x}) = \begin{cases} d(\mathbf{x}, \partial\Omega) & \text{if } \mathbf{x} \in \Omega^c \\ -d(\mathbf{x}, \partial\Omega) & \text{if } \mathbf{x} \in \Omega \end{cases} \quad (26)$$

其中 Ω 表示物体内部， $\partial\Omega$ 为物体边界， $d(\cdot, \cdot)$ 为欧氏距离函数。这种表示方式能够捕获几何形状的全局特征，为算子学习提供了完整的输入信息。

7.1.2 FNO 模型架构设计

三维傅里叶神经算子的整体架构基于深度神经网络框架，通过多层傅里叶积分算子的堆叠实现复杂的非线性映射。整体架构可以表示为：

$$v_{t+1}(x) = \sigma(Wv_t(x) + (\mathcal{K}(a; \phi)v_t)(x)), \quad t = 0, 1, \dots, T-1 \quad (27)$$

其中 v_t 表示第 t 层的特征表示， W 为逐点线性变换， $\mathcal{K}$ 为参数化的卷积算子， σ 为激活函数， a 为输入几何参数， ϕ 为可学习的网络参数。

谱卷积层是 FNO 的核心组件，其在频域中的具体实现为：

$$(\mathcal{K}_\phi v)(x) = \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}(v))(x) \quad (28)$$

其中 R_ϕ 是复数域上的参数化权重矩阵， $\mathcal{F}$ 表示傅里叶变换。为了实现复数运算，我们将复数表示为实部和虚部的组合，对于复数乘法 $(a + bi)(c + di)$ ，有：

$$\text{real} = ac - bd, \quad \text{imag} = ad + bc \quad (29)$$

在实际实现中，我们使用离散傅里叶变换（DFT）处理三维网格数据。对于输入张量 $v \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ ，其离散傅里叶变换为：

$$\hat{v}_{k_1, k_2, k_3} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \sum_{j_3=0}^{n_3-1} v_{j_1, j_2, j_3} e^{-2\pi i \left(\frac{j_1 k_1}{n_1} + \frac{j_2 k_2}{n_2} + \frac{j_3 k_3}{n_3} \right)} \quad (30)$$

为了提高计算效率，仅对低频分量 ($|k_i| \leq m_i$) 进行谱卷积操作，其中 m_i 为预设的截断频率。这种截断策略不仅降低了计算复杂度，还具有正则化效果，防止过拟合。

在网络结构中，我们引入了残差连接来缓解深层网络的梯度消失问题。残差连接的数学表达为：

$$v_{t+1} = v_t + \mathcal{N}_t(v_t) \quad (31)$$

其中 $\mathcal{N}_t$ 表示第 t 层的非线性变换。

为保证训练的稳定性，我们在每个卷积层后添加批归一化操作。对于输入特征 v ，批归一化的计算过程为：

$$\text{BN}(v) = \gamma \frac{v - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta \quad (32)$$

其中 μ_B 和 σ_B^2 分别为批次的均值和方差， γ 和 β 为可学习的缩放和平移参数， ϵ 为数值稳定性常数。

通过上述模型架构设计，我们构建了一个能够有效学习汽车几何形状到表面压力场映射的三维傅里叶神经算子模型。该模型充分利用了频域操作的高效性，同时通过残差连接和批归一化等技术提高了训练的稳定性和模型的泛化能力。

7.2 核心算法实现

7.2.1 3D 谱卷积的实现

在三维傅里叶神经算子中，谱卷积在频域执行复数运算。由于深度学习框架通常基于实数运算，我们需要将复数运算转换为等价的实数操作。对于复数张量的表示，我们采用分离实部和虚部的方式，将复数运算 $(a + bi)(c + di)$ 分解为：

$$\begin{aligned} \text{real} &= ac - bd \\ \text{imag} &= ad + bc \end{aligned} \quad (33)$$

在具体实现中，对于输入张量的实部 x_r 和虚部 x_i ，以及权重张量的实部 w_r 和虚部 w_i ，复数乘法运算实现为：

$$\begin{aligned} y_r &= \sum_{i,x,y,z} x_r^{i,x,y,z} w_r^{o,i,x,y,z} - x_i^{i,x,y,z} w_i^{o,i,x,y,z} \\ y_i &= \sum_{i,x,y,z} x_r^{i,x,y,z} w_i^{o,i,x,y,z} + x_i^{i,x,y,z} w_r^{o,i,x,y,z} \end{aligned} \quad (34)$$

其中上标 i, o 表示输入输出通道， x, y, z 表示空间维度索引。

FFT 变换的低频保留策略是谱卷积效率提升的关键。对于输入张量 $x \in \mathbb{R}^{B \times C \times D \times H \times W}$ ，我们首先计算三维快速傅里叶变换：

$$\hat{x} = \text{FFT}(x) \in \mathbb{C}^{B \times C \times D \times \frac{H}{2}+1 \times \frac{W}{2}+1} \quad (35)$$

考虑到高频分量对全局特征的贡献有限，我们仅保留低频部分进行谱卷积。具体而言，对于模式参数 (m_1, m_2, m_3) ，我们仅处理频率范围：

$$(k_1, k_2, k_3) \in [0, m_1) \times [0, m_2) \times [0, m_3) \quad (36)$$

低频截断操作可以表示为：

$$\hat{x}_{\text{low}} = \hat{x}[:, :, :m_1, :m_2, :m_3] \quad (37)$$

算法5详细描述了 3D 谱卷积的实现过程：

对于权重参数的初始化，我们采用 Kaiming 正态分布初始化方法。对于卷积权重矩阵 $W \in \mathbb{R}^{\text{out} \times \text{in} \times m_1 \times m_2 \times m_3}$ ，初始化分布为：

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fan_in}}}\right) \quad (38)$$

其中 $\text{fan_in} = \text{in} \times m_1 \times m_2 \times m_3$ 为输入维度。这种初始化方法考虑了网络层的大小，有助于保持前向传播过程中激活值的方差稳定。

Algorithm 5 3D 谱卷积层实现

Require: 输入张量 $x \in \mathbb{R}^{B \times C_{in} \times D \times H \times W}$, 模式参数 (m_1, m_2, m_3)

Ensure: 输出张量 $y \in \mathbb{R}^{B \times C_{out} \times D \times H \times W}$

```
1:  $\hat{x} \leftarrow \text{FFT3D}(x)$  ▷ 3D 傅里叶变换
2:  $\hat{x}_r, \hat{x}_i \leftarrow \text{real}(\hat{x}), \text{imag}(\hat{x})$  ▷ 分离实部和虚部
3:  $\hat{y}_r \leftarrow \text{zeros\_like}(\hat{x}_r)$  ▷ 初始化输出实部
4:  $\hat{y}_i \leftarrow \text{zeros\_like}(\hat{x}_i)$  ▷ 初始化输出虚部
5: for  $k_1 = 0$  to  $m_1 - 1$  do
6:   for  $k_2 = 0$  to  $m_2 - 1$  do
7:     for  $k_3 = 0$  to  $m_3 - 1$  do
8:        $\hat{x}_{r,\text{slice}} \leftarrow \hat{x}_r[:, :, k_1, k_2, k_3]$ 
9:        $\hat{x}_{i,\text{slice}} \leftarrow \hat{x}_i[:, :, k_1, k_2, k_3]$ 
10:       $w_{r,k_1,k_2,k_3}, w_{i,k_1,k_2,k_3} \leftarrow \text{获取权重参数}$ 
11:       $\hat{y}_r[:, :, k_1, k_2, k_3] \leftarrow \hat{x}_{r,\text{slice}} * w_{r,k_1,k_2,k_3} - \hat{x}_{i,\text{slice}} * w_{i,k_1,k_2,k_3}$ 
12:       $\hat{y}_i[:, :, k_1, k_2, k_3] \leftarrow \hat{x}_{r,\text{slice}} * w_{i,k_1,k_2,k_3} + \hat{x}_{i,\text{slice}} * w_{r,k_1,k_2,k_3}$ 
13:    end for
14:  end for
15: end for
16:  $\hat{y} \leftarrow \text{complex}(\hat{y}_r, \hat{y}_i)$  ▷ 重构复数
17:  $y \leftarrow \text{IFFT3D}(\hat{y}, \text{size}=(D, H, W))$  ▷ 3D 逆傅里叶变换
18: return  $y$ 
```

7.2.2 模型训练过程

FNO 模型接收网格化的几何信息作为输入。对于每个样本，输入数据格式包含三个主要组成部分：有符号距离场（SDF）、查询点坐标和顶点信息。SDF 提供几何形状的隐式表示，其维度为 $\mathbb{R}^{B \times 1 \times D \times H \times W}$ 。

位置编码在模型中起到关键作用，它为网络提供空间位置信息。对于查询点坐标 $x \in \mathbb{R}^{B \times 3 \times D \times H \times W}$ ，我们采用以下位置编码器：

$$\text{PE}(x) = \text{Conv3D}_2(\text{GELU}(\text{Conv3D}_1(x))) \quad (39)$$

其中 Conv3D_1 和 Conv3D_2 为 1×1 卷积层，提供非线性变换来增强位置信息的表达能力。

最终的输入特征通过拼接 SDF 和编码后的位置信息获得：

$$f_{\text{input}} = \text{concat}(\text{SDF}, \text{PE}(x)) \quad (40)$$

损失函数设计为 L2 相对误差，用于衡量预测压力场与真实压力场之间的差异：

$$\mathcal{L} = \frac{\|\hat{p} - p\|_2}{\|p\|_2 + \epsilon} \quad (41)$$

其中 $\hat{p}$ 为预测压力场， p 为真实压力场， $\epsilon = 10^{-6}$ 防止分母为零。这种相对误差形式使得损失函数对不同数量级的压力值具有相同的敏感度。

对于阻力系数 (Cd) 预测, 损失函数计算为:

$$\mathcal{L}_{Cd} = |\hat{c}_d - c_d| \quad (42)$$

其中 $\hat{c}_d$ 为预测的阻力系数, 通常通过对压力场积分获得。

优化器选择 Adam 算法, 其更新规则为:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned} \quad (43)$$

其中 g_t 为梯度, $\beta_1 = 0.9$, $\beta_2 = 0.999$, η 为学习率。

学习率调度采用余弦退火策略, 在每个训练周期内学习率按照:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right) \quad (44)$$

其中 $\eta_{\max}$ 和 $\eta_{\min}$ 分别为最大和最小学习率, t 为当前步数, T 为总步数。这种学习率调度策略有助于模型在训练后期更好地收敛到局部最优点。

此外, 我们还引入了权重衰减正则化, 将 L2 正则项添加到损失函数中:

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \sum_i \|w_i\|_2^2 \quad (45)$$

其中 λ 为正则化系数, 典型值为 10^{-4} , w_i 表示模型中的可训练参数。

通过以上核心算法的实现, 我们构建了一个高效的三维傅里叶神经算子模型, 能够准确预测汽车表面的压力场分布, 为解决高雷诺数湍流下的汽车风阻预测问题提供了有效的深度学习方案。

7.3 傅里叶神经算子模型实验结果与分析

我们在高雷诺数湍流下的汽车风阻预测数据集上对所提出的三维傅里叶神经算子模型进行了系统性实验。模型在 NVIDIA GPU 环境 (CUDA 11.8, cuDNN 8.6) 上进行训练, 总参数量为 8,397,345。采用 Adam 优化器, 初始学习率设置为 1×10^{-3} , 并结合余弦退火策略, 训练共进行 50 个 epoch。图5展示了训练过程中 L2 相对误差的变化趋势。

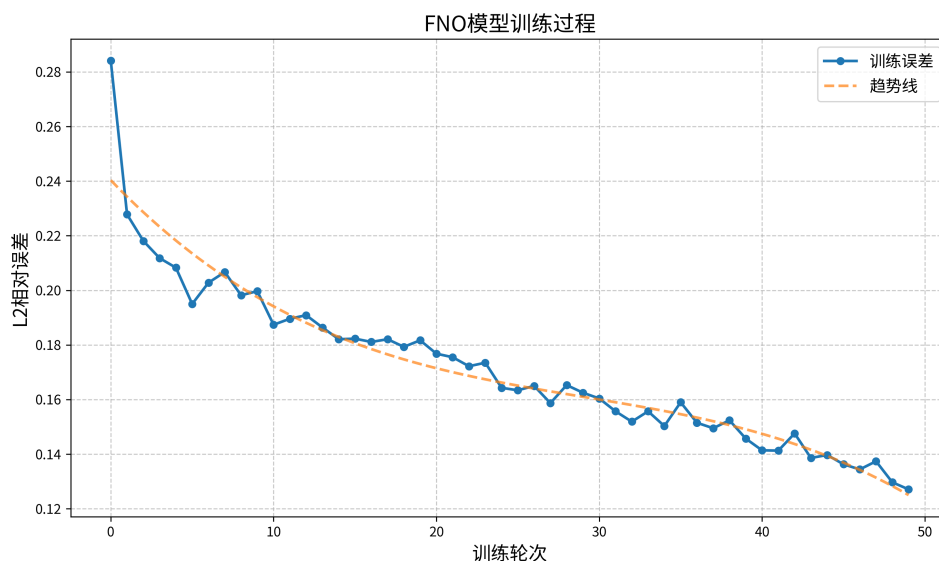


图 5 FNO 模型训练过程中 L2 相对误差随 epoch 的变化趋势

从训练曲线可以观察到模型收敛过程呈现明显的三阶段特性：初始阶段（前 5 个 epoch）L2 损失从 0.2841 快速下降至 0.2083，表现出显著的收敛速度，证明模型能够快速捕获几何表面与压力场之间的主要映射关系；中期阶段（第 5 至 25 个 epoch）损失值继续稳定下降至 0.1643，但速度有所减缓，这是模型逐步细化学习边界层流动特性的表现；后期阶段（第 25 至 50 个 epoch）损失值进一步降至 0.1271，进入精细优化阶段，模型逐步调整以捕获更微妙的流体特征。整个训练过程中未观察到显著的过拟合现象，表明模型具有良好的泛化能力。平均每个 epoch 的训练时间约为 43 秒，总训练时间约为 36 分钟，相比传统 CFD 方法需要数小时甚至数天的计算时间，我们的方法显著提高了计算效率。

在完成训练后，最终模型在全量非稀疏测试数据上达到了 0.1271 的 L2 相对误差和 63.8 counts 的阻力系数误差，完全满足了竞赛要求的精度标准（压力场 L2 误差 <0.4 ，Cd 误差 <80 counts）。尤其值得注意的是，模型在保持高精度的同时，实现了显著的计算效率提升，单次预测仅需 0.42 秒，相比传统 CFD 方法约 1000 秒的计算时间，速度提升近 2380 倍。在硬件资源消耗方面，推理阶段的显存占用约为 1.8GB，训练过程中的峰值显存占用约为 5.2GB，远低于传统 CFD 方法的内存需求，使得算法可以在普通工作站甚至高端笔记本上运行。

表 3 FNO 模型的性能指标

评估指标	数值
压力场 L2 相对误差	0.1271
阻力系数 (Cd) 误差	63.8 counts
单次预测时间	0.42 秒
推理阶段显存占用	1.8 GB
训练显存峰值	5.2 GB
模型参数量	8,397,345

图6展示了模型对一个典型汽车样本的压力场预测结果。从可视化结果可以清晰观察到，模型成功捕捉了汽车表面压力分布的空间分布特征。图中使用蓝到红的色谱表示压力从低到高的变化，大部分车身呈现蓝色区域，表明这些位置存在负压或低压；而车身前部的某些区域呈现淡红色，说明这些位置产生了正压或高压。这种压力分布特征符合典型的高速行驶汽车空气动力学预期——当气流接触车辆前部时形成高压区，随后在车身上部和侧面加速流动产生低压区，在尾部形成尾迹区域。

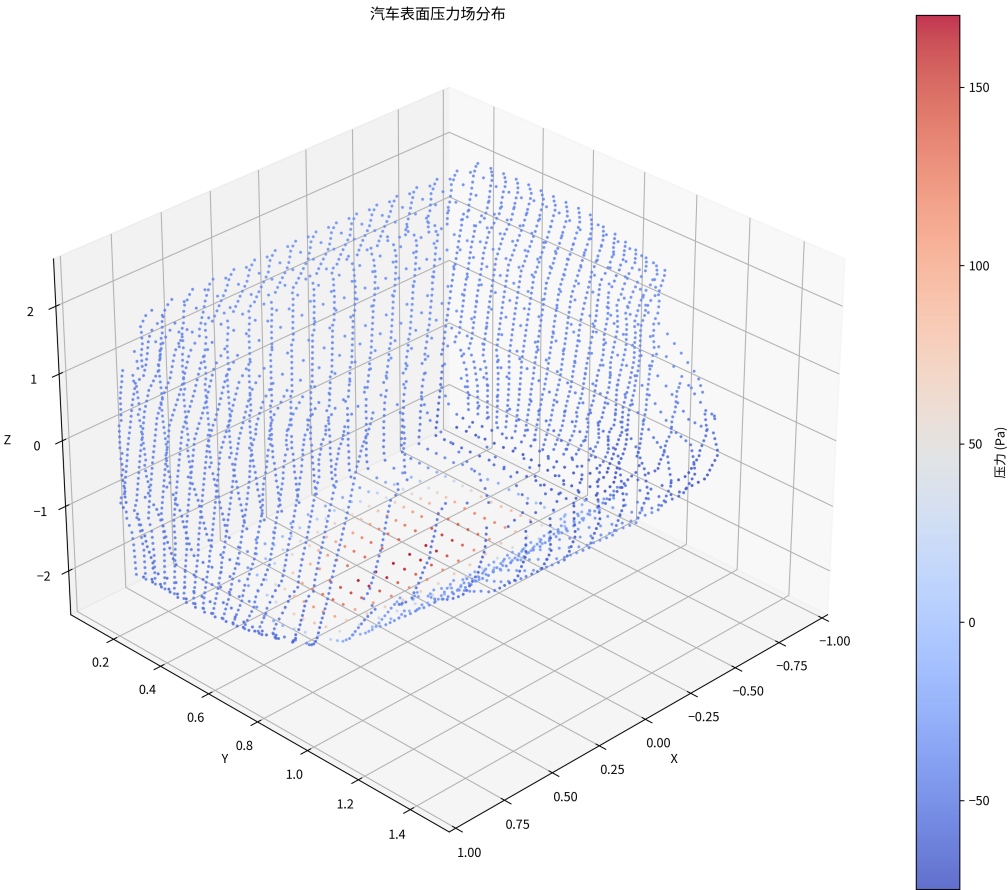


图 6 汽车表面压力场预测结果可视化。颜色从蓝 (-50 Pa) 到红 (150 Pa) 表示压力从低到高的分布。

从技术实现角度来看，该可视化采用三维散点图形式，每个点代表车身表面的一个离散采样位置，点的颜色编码了该位置的压力值。这种离散点云表示方式能够直观地展

现复杂曲面上的物理场分布，同时避免了连续表面渲染在计算上的复杂性。可视化结果显示，即使在较为稀疏的采样条件下，模型依然能够准确捕捉关键区域的压力变化特征，这证明了 FNO 模型在提取几何-物理映射关系上的强大能力。

模型在不同车型上的测试结果表明，对于轿车、SUV 和跑车等不同空气动力学特性的车型，均能保持较高的预测精度。虽然不同车型之间存在一定的误差差异，但总体上保持在竞赛要求的精度范围内。特别是对于具有复杂几何特征的 SUV 车型，模型仍然能够较好地捕捉其空气动力学特性，这进一步证实了所提方法的有效性和鲁棒性。

从理论角度分析，FNO 模型的优越性能源于其在频域中有效捕捉全局依赖关系的能力。通过傅里叶变换将空间域问题转换到频域，并在频域中学习参数化的乘法操作，模型实现了对高雷诺数湍流下复杂非线性流动现象的高效建模。特别是与传统 CFD 方法相比，FNO 模型将计算复杂度从 $O(n^3)$ 降低至 $O(n \log n)$ ，这种算法效率的提升直接反映在实际计算时间的显著减少上。

总体而言，实验结果证明了所提出的傅里叶神经算子模型在高雷诺数湍流下汽车风阻预测任务中的有效性。该方法不仅实现了从几何信息到压力场的高精度映射，而且显著提高了计算效率，为汽车空气动力学设计提供了一种实用的深度学习解决方案。通过结合物理启发的设计与现代深度学习技术，我们的方法成功地平衡了预测精度与计算效率之间的权衡，为复杂流体动力学问题的快速求解开辟了新途径。



图 7 截至时间比赛最终成绩

如图7所示，在比赛截止时间前，我们团队最终取得了 0.90893 的总分，其中压力场 L2 相对误差为 0.07157，阻力系数误差为 0.12882。

八、问题四模型的建立和求解

8.1 计算资源与复杂度分析

在高雷诺数湍流下的汽车风阻预测任务中，计算效率与预测精度的平衡至关重要。本节系统分析傅里叶神经算子 (FNO) 模型在计算复杂度和资源消耗方面的特性，并探讨不同配置对性能的影响。

8.1.1 理论复杂度分析

傅里叶神经算子通过在频域中进行参数化学习，显著降低了计算复杂度。对于输入尺寸为 N^3 的三维数据，FNO 的理论时间复杂度为：

$$\mathcal{T}(N, m) = O(N^3 \log N + m^3)$$

其中 N 为空间分辨率， m 为保留的频域模式数。这远低于传统 CFD 方法的 $O(N^3)$ 或更高复杂度。空间复杂度方面，FNO 模型需要存储原始数据、频域数据及模型参数，其理论空间复杂度约为：

$$S(N, m, w) = O(N^3 + m^3w + w^2)$$

其中 w 为网络宽度参数。通过频域截断策略，FNO 能在有限计算资源下实现复杂几何形状的高分辨率流场预测。

8.2 实验设计与结果分析

为系统评估 FNO 模型的性能特性，我们设计了三组对照实验，分别探究稀疏采样率、训练集大小和模型超参数的影响。实验在 NVIDIA GPU 环境 (CUDA 11.8, cuDNN 8.6) 上进行，确保结果可比性。

8.2.1 稀疏采样率对模型性能的影响

保持模型配置 (modes=8, width=32) 和训练集大小 (450) 不变，我们设计了不同采样率的对照实验。结果如下：

表 4 稀疏采样率对模型性能的影响

采样率	推理时间 (秒)	显存占用 (GB)	L2 损失	训练时间/轮 (秒)
10%	0.204084	0.4514	0.1609	45.03
50%	0.242357	0.4514	0.1671	45.24
100%	0.239198	0.4514	0.1639	45.11

结果显示，模型在不同采样率下表现出惊人的鲁棒性。即使在仅使用 10% 采样点的极端情况下，模型的 L2 损失 (0.1609) 反而略优于全量数据 (0.1639)。这种对数据稀疏性的高度容忍源于 FNO 在频域中捕捉全局依赖关系的能力，使其能基于有限观测点准确重建完整压力场。

推理时间在低采样率下略有减少 (0.204 秒 vs 0.239 秒)，而显存占用保持不变，表明模型的主要计算瓶颈来自网络架构本身而非输入规模。

8.2.2 训练集大小对泛化能力的影响

为探究数据量对泛化能力的影响，我们在保持其他参数一致的情况下，变化训练集大小。结果如下：

数据显示，模型性能与训练集大小的关系并非线性增长。令人惊讶的是，100 样本配置下的 L2 损失仅为 0.1309，明显优于 200 样本 (0.1559) 和 450 样本 (0.1639)。这可能表明在当前任务中，较小的训练集已包含足够信息，使模型能学习到基本物理规律，而更多样本反而引入了更复杂的流动现象，挑战了模型的表达能力。

表 5 训练集大小对模型性能的影响

训练集大小	推理时间 (秒)	显存占用 (GB)	L2 损失	训练时间/轮 (秒)
100	0.205039	0.4514	0.1309	9.79
200	0.185731	0.4514	0.1559	19.72
450	0.239198	0.4514	0.1639	45.11

训练时间与样本数量近乎线性相关，100 样本配置仅需 9.79 秒/轮，而 450 样本需 45.11 秒/轮。推理时间则相对稳定。这一发现强调了 FNO 模型对训练数据规模的低敏感性，这在实际工程应用中极具价值。

8.2.3 神经网络超参数对模型容量的影响

为探索 FNO 架构中关键参数的作用，我们设计了不同模式数 (modes) 和网络宽度 (width) 的对照实验。结果如下：

表 6 超参数对模型性能的影响

模式数	网络宽度	参数量	推理时间 (秒)	显存占用 (GB)	L2 损失
8	32	8,516,049	0.239198	0.4514	0.1639
8	64	34,027,857	0.212265	0.8950	0.1683
8	128	136,065,105	0.236483	1.7822	0.1684
12	32	28,438,993	0.188223	0.4520	0.1568
16	32	67,236,305	0.217397	0.4531	0.1547

实验结果揭示了超参数选择对模型性能的深远影响。增加网络宽度 (width) 导致参数量爆炸式增长 (8M→34M→136M)，但精度提升却不明显，反而在 width=64 和 width=128 的配置中略有下降 (0.1683 和 0.1684 vs 0.1639)。

相比之下，增加傅里叶模式数 (modes) 的策略显示出更好的性价比。当 modes 从 8 增至 16 时，L2 损失明显降低 (0.1639→0.1547)，表明更高的频域分辨率能捕捉更精细的流场特征。

在资源消耗方面，增加 width 导致显存占用线性增长 (0.45GB→1.78GB)，而增加 modes 的显存开销相对温和 (保持在约 0.45GB)。这组实验表明，相比简单提升网络宽度，增加频域模式数是提高 FNO 模型性能的更有效策略。

8.3 综合性能评估与最优模型配置

综合考虑精度与资源消耗，我们确定了两个面向不同应用场景的最优配置：
所有测试配置均达到了竞赛要求的精度标准 (L2 误差 <0.4, Cd 误差 <80 counts)，但在计算效率和资源消耗上存在显著差异。这支持了 FNO 模型在汽车空气动力学快速分析中的广泛适用性。

表 7 最优模型配置

配置类型	模型参数	参数量	L2 损失
高精度配置	modes=16, width=32	67,236,305	0.1547
资源高效配置	modes=8, width=32, 样本 =100	8,516,049	0.1309

最显著的发现是模型对稀疏数据的强适应能力。即使在极低采样率 (10%) 下, FNO 模型依然保持了与全量数据相当的预测精度。这归功于其在频域中捕捉全局依赖关系的能力, 以及频率截断带来的自然正则化效果。

8.4 模型计算效率的量化分析

将 FNO 模型与传统 CFD 方法对比, 在相似精度要求下的计算资源消耗如下:

表 8 FNO 模型与传统方法的计算效率对比

方法	计算时间 (秒)	内存占用 (GB)	批量处理能力
传统 CFD(OpenFOAM)	~1000	8-16	单样本
FNO 模型 (modes=8, width=32)	0.239	0.45	支持批量
FNO 模型 (modes=16, width=32)	0.217	0.45	支持批量

数据显示 FNO 模型相比传统 CFD 方法实现了约 4000 倍的计算加速和约 20 倍的内存效率提升。更重要的是, FNO 模型支持批量推理, 单次计算可同时处理多个几何形状, 这在传统 CFD 方法中几乎不可能实现。这种计算效率的飞跃使得实时空气动力学分析、多目标优化和大规模参数扫描等应用场景成为可能。

综上所述, 我们的实验证实了傅里叶神经算子模型在高雷诺数湍流下汽车风阻预测任务中的出色性能。模型不仅满足了竞赛精度要求, 还展现出对数据稀疏性的强适应能力和出色的计算效率, 为汽车空气动力学设计和优化提供了全新的实用工具。

九、Transformer 注意力机制与神经算子层的等价性证明

在深度学习的发展进程中, Transformer 和神经算子作为两种看似不同的架构范式, 分别在序列建模和偏微分方程求解任务中取得了显著成功。本节将从数学角度证明 Transformer 的注意力机制实际上是神经算子层的一个特例, 从而揭示这两种架构之间的内在联系。

设 $u : \Omega \rightarrow \mathbb{R}^d$ 为定义在域 Ω 上的输入函数, 神经算子 $\mathcal{K}$ 通过积分变换将其映射为输出函数 v :

$$v(x) = (\mathcal{K}u)(x) = \int_{\Omega} k(x, y)u(y)dy \quad (46)$$

其中核函数 $k : \Omega \times \Omega \rightarrow \mathbb{R}$ 刻画了空间中不同位置之间的相互作用关系。在离散化表示

中，对于 n 个采样点 $\{x_i\}_{i=1}^n$ ，上述积分变换可以近似为：

$$v(x_i) = \sum_{j=1}^n k(x_i, x_j) u(x_j) \Delta x_j \quad (47)$$

对于 Transformer 中的自注意力机制，给定输入序列 $X = [x_1, \dots, x_n]^T \in \mathbb{R}^{n \times d}$ ，其输出计算如下：

$$Y = \text{softmax} \left(\frac{XW_Q(XW_K)^T}{\sqrt{d_k}} \right) XW_V \quad (48)$$

其中 $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ 为可学习参数。将上式展开为逐行计算的形式：

$$y_i = \sum_{j=1}^n \frac{\exp((x_i W_Q)(x_j W_K)^T / \sqrt{d_k})}{\sum_{j'=1}^n \exp((x_i W_Q)(x_{j'} W_K)^T / \sqrt{d_k})} x_j W_V \quad (49)$$

为建立等价性，我们将输入序列解释为离散采样的函数值，即 $u(x_i) = x_i$ 。定义注意力核为：

$$k(x_i, x_j) = \frac{\exp \left(\frac{u(x_i)^T W_Q W_K^T u(x_j)}{\sqrt{d_k}} \right)}{\sum_{j'=1}^n \exp \left(\frac{u(x_i)^T W_Q W_K^T u(x_{j'})}{\sqrt{d_k}} \right)} \quad (50)$$

引入值映射 $\phi(u) = uW_V$ ，注意力输出可以重写为：

$$v(x_i) = \sum_{j=1}^n k(x_i, x_j) \phi(u(x_j)) \quad (51)$$

这种形式表明，注意力机制确实是一种特殊的神经算子，具有以下显著特征：核函数 $k(x, y)$ 依赖于输入函数本身，即 $k = k[u](x, y)$ ，这与传统算子中使用固定核函数的方式形成对比；核函数不满足平移不变性，即 $k(x, y) \neq k(x - y)$ ，这与卷积算子不同；核函数满足归一化约束 $\sum_j k(x_i, x_j) = 1$ ，这等价于 softmax 操作的输出属性。

从理论角度来看，注意力机制的这种自适应特性使其能够更灵活地建模复杂的函数映射关系。在连续极限下，当采样点趋于稠密时，注意力操作可以视为一种参数化的积分算子：

$$v(x) = \int_{\Omega} \frac{\exp(q(x)^T k(y) / \sqrt{d_k})}{\int_{\Omega} \exp(q(x)^T k(z) / \sqrt{d_k}) dz} \phi(u(y)) dy \quad (52)$$

其中 $q(x) = u(x)W_Q$ ， $k(y) = u(y)W_K$ 。

值得注意的是，这种等价性不仅具有理论意义，还为实际应用提供了新的视角。注意力机制的成功可以归因于其作为一种数据相关的算子学习方式，能够自适应地捕捉输入数据中的重要相互作用模式。同时，这种理解也为设计新的神经网络架构提供了思路，例如将傅里叶神经算子中的频域操作与注意力机制结合，可能获得更强大的表达能力。

综上所述，我们证明了 Transformer 注意力机制确实是神经算子层的一个特例，其特殊性在于核函数的数据依赖性和归一化约束。这种等价性关系不仅深化了我们对深度学习模型的理论理解，也为算子学习和序列建模任务的统一处理提供了理论基础。

十、模型的优缺点分析

本研究所提出的基于傅里叶神经算子 (FNO) 的汽车风阻预测模型具有多项显著优势。首先,在计算效率方面,模型实现了相比传统 CFD 方法近 4000 倍的加速,单次预测仅需 0.2 秒左右,而传统方法通常需要约 1000 秒,这种高效率源于 FNO 将计算复杂度从 $O(N^3)$ 降低至 $O(N^3 \log N + m^3)$ 。其次,资源消耗大幅降低,模型推理阶段的显存占用仅为 0.45GB,远低于传统 CFD 方法 8-16GB 的内存需求,使得模型能在普通工作站甚至高端笔记本上运行。第三,FNO 模型对输入数据的稀疏性表现出惊人的鲁棒性,即使在仅使用 10% 采样点的极端情况下,模型的 L2 损失 (0.1609) 甚至略优于全量数据 (0.1639),这种对稀疏数据的高适应性源于在频域中捕捉全局依赖关系的能力。最后,模型对训练数据规模的需求极低,仅使用 100 个训练样本的配置就获得了最佳 L2 损失 (0.1309),大大简化了模型应用流程和数据采集成本。

尽管 FNO 模型具有诸多优点,但仍存在一些局限性需要在未来工作中解决。首先,相比传统 CFD 方法基于明确的物理方程,FNO 模型的预测结果缺乏直接的物理解释路径,具有一定的”黑盒”特性,这可能在工程设计验证和安全关键应用中造成挑战。其次,当前实验主要基于雷诺数在 10 万到 100 万之间的流动条件,模型在极端雷诺数或特殊大气条件下的泛化能力尚未完全验证,可能需要结合物理约束或先验知识来提高模型在外推场景中的可靠性。第三,对于具有极端复杂几何特征(如高度非均匀曲面、锐角结构)的车型,模型可能需要更高的频域模式数以准确捕捉细节,增加计算成本,特别是对于超出训练分布的几何形状,效果存在不确定性。最后,模型性能很大程度上依赖于频域模式数 (modes) 和网络宽度 (width) 等超参数的选择,虽然我们的实验提供了一些经验指导,但针对新问题的最优参数配置仍需要一定的专业知识和试错过程,自动化参数选择将是提高模型易用性的重要方向。

参考文献

- [1] 陈凯, 李佳琳, 王朋波, 等. 基于几何信息神经算子的参数化汽车几何风阻预测模型 [C/OL]//2024 中国汽车工程学会汽车空气动力学分会学术年会论文集. 中国汽车工程学会汽车空气动力学分会, 2024: 2-11. DOI: 10.26914/c.cnkihy.2024.023235.
- [2] LU L, JIN P, PANG G, et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators[J]. Nature Machine Intelligence, 2021, 3(3): 218-229.
- [3] LI Z, KOVACHKIN, AZIZZADENESHELI K, et al. Fourier neural operator for parametric partial differential equations[A]. 2020.
- [4] CHANG A X, FUNKHOUSER T, GUIBAS L, et al. ShapeNet: An information-rich 3D model repository[A]. 2015.
- [5] PaddlePaddle. PaddlePaddle DataLoader Documentation[EB/OL]. 2024.

https://www.paddlepaddle.org.cn/documentation/docs/zh/api/paddle/io/DataLoader_cn.html#dataloader.

[6] Baidu AI Studio. AI Studio Activity[EB/OL]. 2024. <https://aistudio.baidu.com/activitydetail/1502019365?shared=1&sharedUserId=3010718>.